LLM-ASSISTED PREDICTIVE MAINTENANCE
OF POWER PLANT BOILERS

Anomaly Detection, Retrieval-Augmented Generation,
and Local Large Language Model Support

M.Tech Project Report

Operations Research and Data Analytics

Indian Institute of Technology Kharagpur

Student: _____

Roll Number: _____

Supervisor: _____

November 2026

Abstract

This project develops an academic prototype for boiler-related anomaly triage and maintenance decision support. The system combines multivariate time-series analysis with retrieval-augmented generation (RAG) and a local large language model (LLM). The numerical pipeline audits a public sensor dataset, aggregates five-second observations to one-minute intervals, establishes chronological training/validation/test partitions, and compares statistical and multivariate detectors. A soft-sensor approach based on Ridge regression, PCA, and an EWMA-based detector are evaluated using controlled synthetic scenarios because the supplied dataset contains no verified fault labels. A document pipeline extracts text from boiler manuals, divides it into overlapping chunks, embeds the chunks using Sentence Transformers, and indexes them in FAISS. Retrieved passages are supplied to a local Qwen2.5-1.5B-Instruct model to draft a cautious maintenance-support report. On the 24 injected scenarios in the final checked package, the Soft Sensor detected 13 events (54.17%), compared with 7 (29.17%) for PCA and 9 (37.50%) for SS-EWMA. These figures measure sensitivity to the selected synthetic patterns only; they do not establish real-world boiler-failure prediction accuracy. The generated report explicitly avoids assigning physical meanings to unverified sensor tags and requires qualified human review.

Keywords: boiler maintenance, anomaly detection, soft sensors, PCA, EWMA, FAISS, RAG, large language models, decision support

Table of Contents

1. Introduction

2. Problem Definition, Aim and Objectives

3. Literature Review: Eight Approaches

4. Dataset Description and Audit

5. Proposed System Architecture

6. Methodology and Algorithms

7. Software Implementation and Coding Instructions

8. Experimental Setup and Results

9. RAG and LLM Demonstration

10. Discussion

11. Limitations, Safety and Threats to Validity

12. Conclusion and Future Work

References

Appendix A. Reproducibility Checklist

1. Introduction

Boiler systems operate through interacting process variables and equipment subsystems. Sensor measurements can help operators identify unusual operating behaviour, but a deviation in one or more tags does not by itself prove a particular equipment fault. Changes in operating regime, sensor drift, noise, instrumentation issues and process interactions may all produce unusual patterns.

Conventional machine-learning methods can analyse numerical sensor patterns, while an LLM can help convert retrieved technical material into readable summaries. The project therefore separates the numerical detection stage from the document-grounded explanation stage. The detector produces an anomaly flag and supporting measurements; the RAG/LLM component retrieves general maintenance references and drafts a report for human review.

The implementation is intentionally an academic prototype. It is not connected to a live plant, does not issue control commands, and does not autonomously diagnose or certify faults.

2. Problem Definition, Aim and Objectives

2.1 Problem statement

How can boiler-related time-series measurements and maintenance documents be combined to identify unusual operating windows and produce traceable maintenance-support notes when verified fault labels, maintenance logs and an authoritative sensor-tag dictionary are unavailable?

2.2 Aim

To develop and evaluate a reproducible anomaly-triage prototype that combines multivariate sensor analysis, document retrieval and a local LLM.

2.3 Objectives

• Audit the sensor dataset for timestamp continuity, missing values, duplicates and basic statistical structure.

• Prepare chronological training, validation and test partitions without randomly shuffling time-series observations.

• Compare baseline anomaly detectors and selected multivariate detectors using controlled injected scenarios.

• Generate episode-level summaries that distinguish injected scenarios from natural windows.

• Create a searchable collection of boiler-related manuals using text extraction, embeddings and FAISS.

• Use a local LLM to draft cautious, cited maintenance-support reports from episode summaries and retrieved passages.

• Document limitations, reproducibility requirements and the need for qualified human review.

3. Literature Review: Eight Approaches

The following approaches represent complementary elements of predictive-maintenance systems. They are not eight mutually exclusive algorithms: some detect numerical deviations, while others retrieve knowledge or support decision-making.

3.1 Threshold and rule-based monitoring

Engineering limits, statistical control limits and rolling-window rules can flag measurements outside expected ranges. They are transparent and useful as baselines. However, fixed limits can generate alerts during normal load changes and may fail to capture multivariate relationships. This project does not invent engineering safety limits because the tag dictionary and operating envelopes are unavailable.

3.2 Conventional machine learning

Unsupervised methods such as Isolation Forest, PCA-based scoring and robust statistical methods can identify unusual patterns without labelled faults. Supervised models such as Random Forest or XGBoost require reliable target labels. The project uses unsupervised detection because the supplied CSV contains no verified fault labels.

3.3 Deep time-series models

LSTM/GRU networks, temporal convolutional networks and transformer models can learn temporal dependencies and forecast signals or detect deviations from expected trajectories. They typically require larger datasets, careful tuning and substantial validation. They were not selected for this prototype because the available observation window is short.

3.4 Survival analysis and remaining useful life

Survival models and remaining-useful-life methods estimate time-to-event or degradation horizon. They require event times, censoring information, degradation histories or maintenance records. Those data are not present in the supplied sensor file, so RUL estimation is outside the defensible scope.

3.5 Retrieval-augmented generation

RAG retrieves relevant passages from manuals and supplies them to an LLM at generation time. It can improve traceability by connecting a draft explanation to source material. It does not guarantee correctness: retrieval can miss relevant passages and the model can misinterpret them. Source/page metadata and manual citation review remain necessary.

3.6 Vector-similarity retrieval using FAISS

Text embeddings represent passages as vectors, and FAISS performs similarity search over those vectors. FAISS is a retrieval library, not an anomaly detector or language model. Similarity scores indicate embedding proximity, not truth or engineering relevance.

3.7 Multimodal or multi-source fusion

Sensor readings can be combined with alarms, operator logs, inspection notes and maintenance work orders. Such data can add context, but they must be reliable and time-aligned. The current prototype uses a late-fusion design: numerical episode summaries are paired with separately retrieved manuals. Verified maintenance logs were not available.

3.8 Agentic LLM systems

An agentic system can plan tasks and call tools for retrieval, analysis and workflow execution. It can coordinate complex workflows but introduces additional uncertainty, complexity and safety risks. The project uses a fixed, auditable pipeline rather than autonomous agents.

3.9 Research gap and project contribution

The project does not propose replacing numerical predictive-maintenance methods with an LLM. Its contribution is a prototype that separates three responsibilities: a numerical detector flags unusual windows, document retrieval supplies maintenance context, and an LLM drafts a cited note with explicit uncertainty. A future controlled user study could compare anomaly-only displays with anomaly-plus-RAG/LLM reports for clarity and traceability. Such a comparison has not been completed in the current implementation.

4. Dataset Description and Audit

4.1 Dataset summary

Property	Observed value
Input file	xinan_uncompleted_data.csv
Shape	86,400 rows × 31 columns (timestamp + 30 sensor tags)
Time column	date
Observation period	2022-03-27 14:28:54 to 2022-04-01 14:28:49
Native sampling interval	0 days 00:00:05
Missing values	30 readings; affected columns: YJJWSLL.AV_0#
Duplicate timestamps	0 reported by the dataset audit
Exact duplicate rows	0 reported by the dataset audit
Fault labels	No label-like columns identified
Distinct sensor tags	30

The observed file contains 86,400 rows and 30 numeric tags plus a timestamp. The time span is approximately five days and the dominant interval is five seconds. The audit identified 30 missing readings in YJJWSLL.AV_0#; this is approximately 0.001% of sensor cells. No label-like fault column was identified.

4.2 Data-quality observations

• The time series has a regular five-second cadence in the audit output, with no detected timestamp gaps above the audit threshold.

• Several tags contain predominantly negative values. These must not be clipped or set to zero without knowing whether the measurements are gauge, differential or transformed values.

• PTCA_8322A.AV_0# and PTCA_8324.AV_0# have relatively coarse resolution and are used as context variables in the pipeline.

• The audit reports a median lag-1 autocorrelation of approximately 0.99, indicating strong temporal dependence.

• Thirteen principal components explain approximately 95% of variance in the standardised sensor matrix.

• The dataset card refers to an approximately 8.6% abnormal fraction in source comments, but the definition and label-generation procedure are not documented. This value is not treated as verified ground truth.

4.3 Data limitations

The exact dataset download URL, licence and authoritative sensor-tag dictionary were not present in the supplied project files. These details must be recovered from the original download page before formal publication. Sensor tag names are preserved as codes in this report; no physical meaning or unit is inferred from the tag names.

5. Proposed System Architecture

The implemented workflow consists of two linked but separately validated branches:

Stage	Input	Operation	Output
Numerical branch	Timestamped sensor CSV	Audit, resample, split, fit detectors, score observations	Anomaly scores, alert episodes, contributing tags
Knowledge branch	Boiler manuals and references	PDF text extraction, chunking, embedding and FAISS indexing	Retrieved passages with source and page metadata
Integration	Episode summary + retrieved passages	Prompt local Qwen model with strict grounding instructions	Draft maintenance-support report
Human review	Draft report and source documents	Check tag meanings, citation relevance and engineering suitability	Reviewed academic example; no autonomous action

The final code uses Python, pandas, NumPy, scikit-learn, Sentence Transformers, FAISS, pypdf and Hugging Face Transformers. The LLM is Qwen2.5-1.5B-Instruct and is loaded locally. A Streamlit dashboard was discussed in the early proposal but is not present in the final consolidated implementation and is therefore not claimed as completed.

6. Methodology and Algorithms

6.1 Data preparation

The CSV is parsed using the `date` timestamp column and sorted chronologically. Numeric sensor columns are selected. The data are resampled to one-minute intervals using the median, followed by forward filling for at most three consecutive intervals. The two context tags are PTCA_8322A.AV_0# and PTCA_8324.AV_0#. Four indicator tags are retained for plotting and are not used as model inputs.

The 7,201 one-minute observations are split chronologically: 70% for training, 15% for validation and the remaining 15% for testing. Based on the earlier execution summary, this corresponds to 5,040 training rows, 1,080 validation rows and 1,081 test rows. Thresholds are selected using validation data rather than the test period.

6.2 Context-conditioned residual view

A linear regression model is fitted on the training partition to estimate each monitored sensor from the two context variables. The residual for sensor j at time t is the observed value minus its context-model prediction. Both the raw sensor view and residual view are standardised using training-derived transformations. The residual view is intended to highlight deviations from relationships learned under the training operating context.

6.3 Baseline detectors

Median absolute deviation (MAD)

A robust per-feature score is computed from deviation from the training median divided by a scaled median absolute deviation. The maximum absolute feature score is used as the observation-level score.

Isolation Forest

An ensemble of 300 isolation trees is fitted to the training feature view. The negative sample score is used as an anomaly score; larger values indicate more unusual observations.

Principal component analysis (PCA)

PCA retains enough components to explain 95% of training variance. The implementation combines Hotelling's T^2 score in retained component space and squared prediction error (SPE) in the reconstruction residual. The maximum normalised score is used.

6.4 Soft Sensor detector

The soft-sensor method models each monitored sensor using all other monitored sensors and the context variables. For each target sensor, a Ridge regression model is trained with alpha = 1.0. A five-fold time-series split with a 30-minute gap is used to obtain out-of-fold residuals on the training data. The robust residual scale is estimated using 1.4826 times the median absolute deviation. The resulting standardised residuals are combined using the maximum absolute residual across monitored sensors to form the observation-level anomaly score.

This approach can flag a sensor whose value is inconsistent with the relationships among the remaining variables. It does not establish why the inconsistency occurred; sensor faults, process changes and unmodelled operating conditions may all produce large residuals.

6.5 SS-EWMA detector

The EWMA transformation smooths residual signals over time. The smoothing parameter is lambda = 0.1, which provides memory across multiple one-minute observations. The smoothed residual score is scaled using robust statistics derived from training out-of-fold residuals. This detector is compared with PCA and the Soft Sensor on the synthetic scenario benches.

6.6 Threshold and persistence rule

The target validation alert rate is 1%, corresponding to a 99th-percentile threshold. A persistent alert is defined using a five-minute window: at least 80% of observations in the window must exceed the threshold. This reduces isolated one-minute spikes but can increase detection delay.

6.7 Synthetic scenario generation

Three benchmark benches are generated, each containing eight injected events, for 24 injected events in total. The scenario types are drift, group shift, noise burst, offset and stuck-sensor behaviour. Scenario durations and magnitudes are set in the pipeline configuration. These are controlled perturbations added to the test data; they are not historical faults or verified physical failures.

6.8 Episode generation

The episode table contains 34 windows: 24 injected episodes and 10 natural windows. The main episode detector is pre-declared as the Soft Sensor. Each row retains an episode ID, type, time window, duration, detected status, scenario metadata where applicable, top variables and context values. Injected and natural windows are kept distinguishable to prevent the LLM from treating synthetic events as actual plant failures.

7. Software Implementation and Coding Instructions

7.1 Software stack

Component	Library / tool	Purpose
Data handling	Python, pandas, NumPy	CSV loading, resampling, statistics and tables
Plots	matplotlib	Diagnostic plots and episode visualisation
Numerical models	scikit-learn, SciPy	Ridge, PCA, Isolation Forest, scaling and metrics
Model persistence	joblib	Saving fitted numerical models
PDF extraction	pypdf	Extract text and page metadata from manuals
Text embeddings	sentence-transformers/all-MiniLM-L6-v2	Convert document chunks and queries to vectors

Component	Library / tool	Purpose
Vector search	FAISS (IndexFlatIP)	Similarity-based retrieval
Local LLM	Transformers + Qwen2.5-1.5B-Instruct	Generate maintenance-support report without an API key

7.2 Notebook execution sequence

• Open `Boiler_Predictive_Maintenance_Final_Consolidated.ipynb` in Google Colab.

• Run the dependency-installation cell in a fresh runtime.

• Upload the original sensor CSV when prompted and confirm that the selected file is the correct dataset.

• Run the numerical pipeline from top to bottom. The pipeline writes audit outputs, benchmark tables, episode CSVs and model files under `outputs/`.

• Inspect the generated episode table and choose an episode ID that exists in the current run.

• Upload the permitted PDF/TXT/MD maintenance references when prompted.

• Build the document index. The first run downloads the sentence-transformer embedding model.

• Run the retrieval test, then generate a report for the selected episode. A GPU runtime is recommended for faster Qwen inference.

• Inspect the retrieved source/page details and manually verify each document-based claim before including it in the final submission.

• Run the packaging cell to save the project outputs and scripts.

7.3 Command-line usage

The numerical script supports the following stages when run from the project directory:

```
python boiler_pipeline.py xinan_uncompleted_data.csv --stage audit
python boiler_pipeline.py xinan_uncompleted_data.csv --stage baselines
python boiler_pipeline.py xinan_uncompleted_data.csv --stage scenarios
python boiler_pipeline.py xinan_uncompleted_data.csv --stage all
python rag_llm.py build
python rag_llm.py query --text "What routine checks are relevant to boiler maintenance?"
python rag_llm.py episode --id N025
```

7.4 RAG implementation details

PDF pages are extracted with pypdf. Each page is split into chunks of 220 words with an overlap of 40 words. Chunks shorter than 80 characters are discarded. Sentence Transformers encodes the chunks using the all-MiniLM-L6-v2 model with vector normalisation. FAISS IndexFlatIP performs inner-product search, equivalent to cosine similarity for normalised vectors. The five highest-scoring non-duplicate passages are retrieved.

The local LLM uses Qwen/Qwen2.5-1.5B-Instruct through Hugging Face Transformers. The prompt prohibits inference of physical sensor meanings, root causes, unprovided readings, control changes and unsupported diagnoses. It asks for observed evidence, possible interpretations, document evidence, inspection questions and an uncertainty note.

7.5 Coding and reproducibility notes

• Use the original CSV and preserve it unchanged.

• Do not randomly shuffle the time-series observations.

• Do not fit preprocessing transformations using validation or test data.

• Keep benchmark runs from different configurations separate.

• Record the exact notebook, script and dataset versions used for each reported result.

• Rebuild the FAISS index if the manual corpus changes.

• Use a fresh runtime for final validation and record any package/model download failures.

8. Experimental Setup and Results

8.1 Baseline synthetic injection tests

The pipeline also evaluates raw and context-residual feature views using MAD, Isolation Forest and PCA over burst, drift and offset injection experiments. The mean detection rates in the saved injection summary are:

Feature view	Detector	Mean detection rate across saved injection cases
Raw	IFOREST	0.00%
Raw	MAD	2.59%
Raw	PCA	44.81%
Residual	IFOREST	0.00%
Residual	MAD	12.22%
Residual	PCA	54.07%

The residual PCA view has the highest mean detection rate among these baseline combinations in the saved injection summary. These values aggregate controlled injection cases and should not be interpreted as real-fault classification accuracy.

8.2 Three-bench detector comparison

Detector	Detected injected events	Detection rate	Median delay among detected
PCA	7/24	29.17%	17.0 min
Soft Sensor	13/24	54.17%	9.0 min
SS-EWMA	9/24	37.50%	7.0 min

The Soft Sensor detected 13 of 24 injected events (54.17%), the highest count among the three detectors in this saved benchmark. PCA detected 7 events (29.17%), and SS-EWMA detected 9 (37.50%). The median detection delays among events detected by each method are reported only over detected events; missed events have no detection delay.

8.3 Detection by injected scenario

Scenario	Events	PCA	Soft Sensor	SS-EWMA
drift	6	50.0%	66.7%	33.3%
group_shift	3	33.3%	100.0%	66.7%
noise_burst	3	33.3%	33.3%	0.0%
offset	6	33.3%	33.3%	33.3%
stuck	6	0.0%	50.0%	50.0%

The Soft Sensor detected all group-shift events in this benchmark (3 of 3), while its detection rate was lower for the other scenario types. None of the three methods detected every injected event. The benchmark is small and the rates are sensitive to the selected scenario magnitudes, durations, thresholds and persistence rule.

8.4 Episode table composition

Episode class	Count
injected_detected	13
injected_missed	11
natural_unflagged_window	9
detected_natural	1

The 34-row episode set contains 13 injected episodes marked as detected, 11 injected episodes marked as missed, nine natural unflagged windows and one detected natural episode. This distinction is important: episode generation does not imply that the selected detector detected every injected event.

8.5 Natural anomaly candidate A001

The saved natural alert A001 begins at 2022-04-01 05:32 and ends at 05:41, with a duration of 10 minutes and a peak score of 2.67. Its leading tags in the episode output are FT_8306B.AV_0# (high, z = +3.1), TE_8304.AV_0# (high, z = +1.7), TE_8303.AV_0# (low, z = -1.6), FT_8302.AV_0# (low, z = -1.6), and AIR_8301B.AV_0# (high, z = +1.3). The physical meanings of these tags are unverified, so this event is only an investigation candidate and cannot be assigned a physical root cause.

9. RAG and LLM Demonstration

9.1 Document collection and retrieval

The checked archive contains 6 PDF reference documents. The RAG index stores 596 text chunks from extracted document pages. The retrieved passage metadata retain source filenames and PDF page numbers.

The RAG implementation retrieved five passages for episode N025. The saved citation audit reports that all citation identifiers used in the report are valid (5 cited IDs; 0 invalid IDs). This audit checks numbering only, not semantic support or factual correctness.

9.2 N025 report example

The N025 report lists five observed tag deviations: YFJ3_ZD2.AV_0# high (+5.9), YFJ3_AI.AV_0# high (+3.7), TE_8303.AV_0# high (+2.3), TE_8313B.AV_0# low (-1.9), and FT_8306B.AV_0# high (+1.8). The LLM explicitly states that no physical interpretation or root cause can be established without a verified tag dictionary and engineering context.

The retrieved source list includes the Combustion Efficiency Optimization Manual (pages 67 and 35) and the J Series Manual (pages 69, 74 and 79). These passages provide general maintenance context. They do not establish that a specific leak, water-quality issue, pressure problem or other fault occurred during N025.

The report includes general inspection questions concerning leakage, water-level indicators, unusual operating fluctuations and oxygen-scavenger concentrations. Before academic submission, each question must be checked against the exact retrieved passage. In particular, a general boiler fact sheet should not be cited as the source for a specific inspection procedure if a more direct maintenance passage is available.

9.3 RAG evaluation status

A successful archived N025 report and valid citation IDs demonstrate that the RAG/LLM path has been exercised. However, a formal retrieval test set with expected relevant passages, retrieval precision/recall, and a human faithfulness rubric was not supplied. Therefore, retrieval quality and LLM factuality are not quantitatively validated in this report.

10. Discussion

The saved three-bench results indicate that the Soft Sensor is the strongest of the three compared detectors for the selected injected scenarios, both in overall event count and group-shift sensitivity. Its advantage is not uniform across scenario types. The detector misses some drift, noise-burst, offset and stuck-sensor cases, which shows that the chosen score and persistence threshold do not capture every injected pattern.

PCA provides a useful multivariate baseline and detects a different subset of events. SS-EWMA provides temporal smoothing but has the lowest overall detection rate in this particular benchmark. The results do not justify claiming that one detector is universally superior; they support selecting the Soft Sensor as the pre-declared episode-generation method for this prototype.

The RAG/LLM stage adds a readable report format and links the episode to manual passages. Its most important design choice is the explicit restriction against guessing tag meanings or diagnosing a root cause. This restraint is appropriate given the lack of tag metadata. The value of the LLM is therefore in organising observations and document context, not in independently determining equipment condition.

11. Limitations, Safety and Threats to Validity

• No verified historical fault labels or maintenance event records are available; real-world precision, recall, F1-score and fault-detection accuracy cannot be calculated.

• The dataset covers approximately five days. This is insufficient to establish long-term performance across seasons, operating regimes, start-ups, shutdowns or maintenance cycles.

• The sensor tag dictionary and engineering units are unavailable. Physical meanings and operating limits must not be inferred from tag codes.

• Negative values in selected tags may be legitimate depending on the measurement convention; automatic clipping or replacement would be unjustified.

• Synthetic injected patterns are simplified and may not reproduce real fault signatures, correlations, durations or amplitudes.

• Detection rates depend on the validation threshold, scenario design, event placement and persistence rule.

• The N025 report demonstrates one archived example, not a statistically representative evaluation of LLM output.

• Citation-ID validation checks that referenced passage numbers exist. It does not prove that a passage supports the associated claim.

• PDF text extraction may omit information from scanned pages, tables, diagrams or complex layouts.

• The final consolidated notebook does not include a Streamlit dashboard or a live plant integration.

• The prototype must not issue setpoint changes, bypass instructions or autonomous maintenance actions. Any operational decision requires qualified personnel and approved plant procedures.

12. Conclusion and Future Work

A consolidated academic prototype has been implemented for boiler-related anomaly triage and maintenance decision support. The numerical pipeline audits and prepares the sensor stream, compares baseline and multivariate anomaly detectors, evaluates three detectors on 24 injected scenarios, and generates a 34-episode dataset. A separate document pipeline extracts and indexes boiler manuals using embeddings and FAISS, retrieves relevant passages, and supplies them to a local Qwen model for a cautious maintenance-support report.

In the final checked benchmark, the Soft Sensor detected 13 of 24 injected events (54.17%), compared with 7 of 24 for PCA (29.17%) and 9 of 24 for SS-EWMA (37.50%). These are controlled synthetic-test results, not real-world failure-prediction metrics. The project therefore demonstrates a decision-support workflow rather than a validated boiler fault-prediction system.

12.1 Future work

• Obtain the original dataset citation, licence and a verified sensor-tag dictionary with units and engineering ranges.

• Review natural episode plots with domain experts and determine whether the unusual windows correspond to operating-regime changes or instrumentation artefacts.

• Collect labelled historical alarms, work orders and maintenance records if permitted; use them for time-aware external validation.

• Prepare a small expert-labelled retrieval query set and evaluate top-k passage relevance.

• Conduct a human review of generated reports using a rubric for citation correctness, unsupported claims, clarity and usefulness.

• Add a simple dashboard only if it is required by the project evaluation rubric.

References

[1] Prabha et al. (2025). “Predictive Maintenance for Industrial Machinery Using LLM.” Full author list, venue, DOI and page details were not present in the supplied literature-review file; verify from the original publication.

[2] Timm et al. (2026). “Identifying LLM Applications for Predictive Maintenance.” Full bibliographic details require verification from the original publication.

[3] Di Maggio (2025). “Toward Autonomous LLM-Based AI Agents for Predictive Maintenance: State of the Art, Challenges, and Future Perspectives.” Applied Sciences. Verify volume, issue, article number and DOI.

[4] Hatten and Malone (2026). “Leveraging Large Language Models for Predictive Maintenance of In-House Critical Equipment.” RAMS 2026. Verify proceedings details and DOI if available.

[5] “Predictive Maintenance with Large Language Models and Transformer-based Survival Analysis” (IECON 2024). Verify full author list, proceedings details and DOI.

[6] Pavanjit Subash. “Power Plant Data: Steam Turbine & Boiler Metrics.” Dataset listing named in the project materials. Exact URL, licence and download date must be added after checking the original dataset page.

Project reference documents

• J Series Manual.pdf (uploaded project reference; page numbers in the RAG index refer to extracted PDF page positions).

- Combustion_Efficiency_Optimization_Manual_Converted.pdf (text-converted manual used in the RAG corpus).
- doe-hdbk-1018-93vol2.pdf.
- 1_400_Ref_03BoilerWaterTreament_SpiraxSarco.pdf.
- ws-commercial-watersense-at-work_Section_6.5_Boilers.pdf.
- Boiler House Service Contract.pdf.

Appendix A. Reproducibility Checklist

- ■ Original CSV retained unchanged.
- ■ Consolidated notebook and Python scripts included in the final archive.
- ■ Dataset source URL and licence verified and added to the final references.
- ■ Sensor tag dictionary obtained or all unverified tags explicitly identified as unknown.
- ■ Fresh Colab run completed from top to bottom; runtime, package versions and any warnings recorded.
- ■ FAISS index rebuilt from the exact manuals included in the submission.
- ■ At least one RAG report manually checked against the exact cited source pages.
- ■ Benchmark run/configuration recorded; no metrics from separate runs combined.
- ■ Synthetic benchmark results clearly labelled as synthetic.
- ■ Report and presentation use the same final metrics and limitations.